



Module : Apprentissage Automatique

Filière : BDIA'1 — Année universitaire 2025-2026

Chapitre 1

Introduction à l'apprentissage automatique

Prof. Dr Naoual Attaoui

Table des matières

- 1. Introduction**
- 2. Histoire de l'apprentissage automatique**
- 3. Définitions et positionnement**
- 4. Programmation classique vs apprentissage automatique**
- 5. Types d'apprentissage automatique**
- 6. Concepts de base**
- 7. Pipeline d'un projet d'apprentissage automatique**
- 8. Limites et défis**

1. Introduction

L'apprentissage automatique est une branche de l'intelligence artificielle qui permet aux systèmes informatiques d'apprendre automatiquement à partir des données afin d'améliorer leurs performances dans une tâche donnée.

Contrairement aux approches de programmation classiques où les règles sont explicitement définies par un programmeur, les systèmes d'apprentissage automatique construisent un modèle à partir d'exemples afin de réaliser des prédictions ou de prendre des décisions.

Aujourd'hui, l'apprentissage automatique constitue l'un des piliers de la science des données, de l'intelligence artificielle moderne et de nombreuses applications industrielles : santé, finance, cybersécurité, industrie, traitement du langage naturel, vision par ordinateur, éducation, transport, etc.

✉ **Fil conducteur — Spam** : Tout au long de ce chapitre, nous utiliserons la **classification d'emails en spam ou non spam** comme exemple fil conducteur. Ce problème simple et concret permet d'illustrer chaque concept présenté : comment un système peut-il apprendre à distinguer un email indésirable d'un email légitime ?

2. Histoire de l'apprentissage automatique

L'apprentissage automatique est le résultat d'une évolution progressive étroitement liée aux progrès de la puissance de calcul et de la disponibilité des données.

2.1. Des origines aux années 1990

Années 1940–1960 — Les fondations. McCulloch et Pitts proposent le premier modèle mathématique du neurone (1943). Rosenblatt développe le **Perceptron** (1957), l'un des premiers algorithmes d'apprentissage. Arthur Samuel introduit le terme « apprentissage automatique » (1959).

Années 1970–1980 — Consolidation. Développement des arbres de décision, méthodes bayésiennes et algorithmes de clustering. En 1986, Rumelhart, Hinton et Williams redécouvrent la **rétropropagation du gradient**, permettant d'entraîner des réseaux multicouches.

Années 1990 — L'apprentissage statistique. Apparition des **SVM** (Vapnik, 1995), méthodes à noyaux, k -NN, forêts aléatoires et Boosting. Naissance de l'apprentissage automatique moderne, fondé sur la théorie de l'apprentissage statistique et la minimisation du risque empirique.

✉ **Fil conducteur — Spam** : Dès les années 1990, les premiers filtres anti-spam utilisent des méthodes bayésiennes (filtre bayésien naïf) pour classer automatiquement les emails à partir de la fréquence des mots.

2.2. L'ère du Big Data et de l'apprentissage profond (années 2000–2010)

L'explosion des données numériques, l'augmentation de la puissance de calcul et le développement des GPU permettent l'essor de l'**apprentissage profond** (*deep learning*). En **2012**, le réseau **AlexNet** (Krizhevsky et al.) remporte la compétition ImageNet, démontrant la supériorité des réseaux neuronaux profonds pour la reconnaissance d'images.

2.3. L'apprentissage automatique aujourd'hui (années 2010–présent)

En **2017**, Vaswani et al. introduisent l'architecture **Transformer**. Le **mécanisme d'attention** permet au modèle de pondérer l'importance relative de chaque élément de l'entrée, plutôt que de traiter la séquence de manière strictement séquentielle comme le faisaient les RNN et LSTM.

Cette architecture donne naissance à **BERT** (Google, 2018) pour la compréhension du langage et **GPT** (OpenAI, 2018–2023) pour la génération de texte. Le lancement de **ChatGPT** (novembre 2022) marque un tournant dans la démocratisation de l'IA.

Aujourd'hui, l'**IA générative** s'étend au-delà du texte : génération d'images (DALL-E, Stable Diffusion), modèles multimodaux (GPT-4V, Gemini), assistance au code (GitHub Copilot). Les techniques de quantification, distillation et apprentissage fédéré permettent le déploiement sur des appareils à ressources limitées.

✉ **Fil conducteur — Spam** : Les filtres anti-spam modernes utilisent des modèles de deep learning et des Transformers capables de comprendre le contexte sémantique d'un email, bien au-delà de la simple fréquence de mots.

3. Définitions et positionnement de l'apprentissage automatique

3.1. Définition

Arthur Samuel (1959) définit l'apprentissage automatique comme :

« le domaine d'étude qui donne aux ordinateurs la capacité d'apprendre sans être explicitement programmés ».

Tom Mitchell (1997) précise : un programme apprend de l'expérience E par rapport à une tâche T et une mesure de performance P , si sa performance sur T , mesurée par P , s'améliore avec l'expérience E .

✉ **Fil conducteur — Spam** : Pour un filtre anti-spam : T = classifier un email (spam ou non), P = taux de bonne classification (F1-score), E = ensemble d'emails étiquetés par des utilisateurs.

3.2. Positionnement scientifique

L'apprentissage automatique se situe à l'intersection de plusieurs domaines :

Domaine	Rôle
Statistiques	Modélisation probabiliste et inférence
Informatique	Conception d'algorithmes

Domaine	Rôle
Optimisation mathématique	Apprentissage des paramètres
Fouille de données	Extraction de connaissances
Science des données	Analyse globale et valorisation
Intelligence artificielle	Systèmes intelligents

3.3. Position de l'apprentissage automatique dans l'IA

L'apprentissage automatique est une sous-discipline de l'intelligence artificielle, et l'apprentissage profond (*deep learning*) est lui-même une sous-catégorie de l'apprentissage automatique.

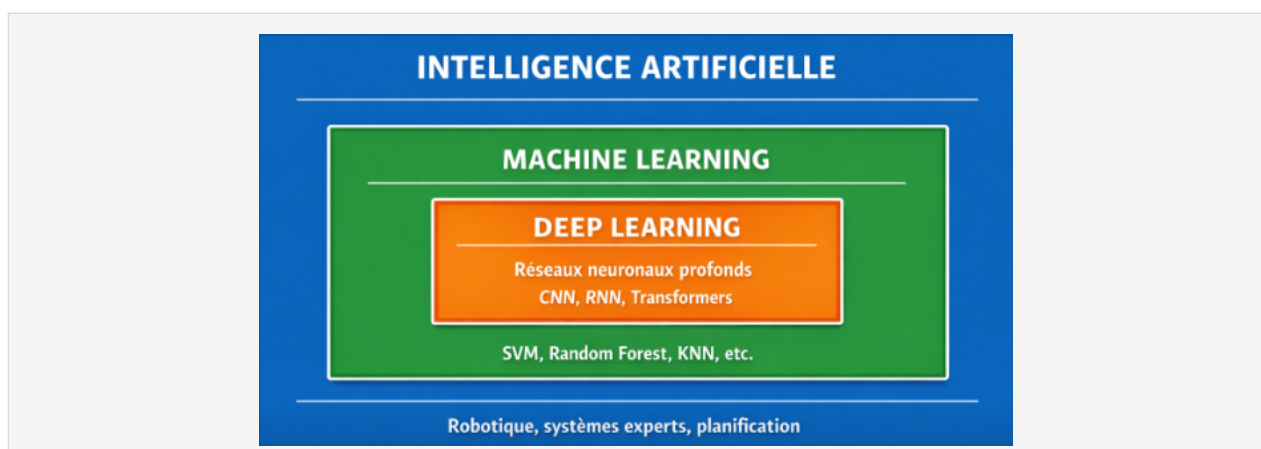


Figure 1 — Positionnement : IA $\supset$ Apprentissage automatique $\supset$ Apprentissage profond

4. Programmation classique vs apprentissage automatique

4.1. Principe de la programmation classique

Le développeur conçoit explicitement l'algorithme qui transforme les entrées en résultats.

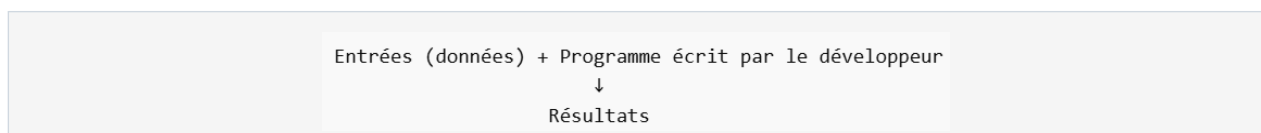


Figure 2 — Schéma de la programmation classique

Avantages :

- comportement prévisible
- interprétation simple
- faible besoin en données

Limites :

- difficile pour des problèmes complexes
- nécessite une formalisation explicite des règles

4.2. Principe de l'apprentissage automatique

L'algorithme apprend automatiquement les règles à partir d'exemples.

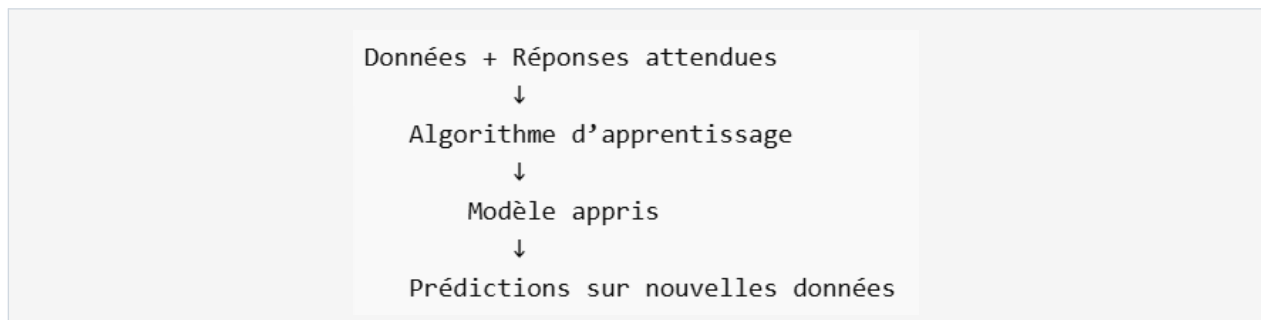


Figure 3 — Schéma de l'apprentissage automatique

L'apprentissage automatique est particulièrement utile lorsque :

1. La formulation algorithmique du problème est difficile ou inconnue.
2. La solution algorithmique existe mais serait trop complexe à programmer explicitement.
3. Le problème évolue avec le temps et nécessite un système adaptatif.
4. Les données disponibles sont très nombreuses.

Avantages :

- capable de traiter des problèmes complexes
- amélioration automatique avec les données
- adaptation aux environnements changeants

Limites :

- dépend fortement de la qualité des données
- interprétation parfois difficile
- coût computationnel élevé

✉ **Fil conducteur — Spam :** En programmation classique, un filtre anti-spam utiliserait des règles manuelles (ex : « si le message contient gagner de l'argent, alors spam »). En apprentissage automatique, on fournit des emails étiquetés et le modèle apprend automatiquement les caractéristiques discriminantes.

4.3. Comparaison

Critère	Programmation classique	Apprentissage automatique
Principe	Règles programmées	Règles apprises des données
Source de connaissance	Expertise humaine	Données

Critère	Programmation classique	Apprentissage automatique
Adaptabilité	Limitée (reprogrammation)	Élevée (réentraînement)
Type de problèmes	Bien définis	Complexes ou mal modélisés

5. Types d'apprentissage automatique

5.1. Apprentissage supervisé

L'algorithme apprend à partir de **données étiquetées** (x_i, y_i) . L'objectif est d'apprendre une fonction $f(x) \approx y$.

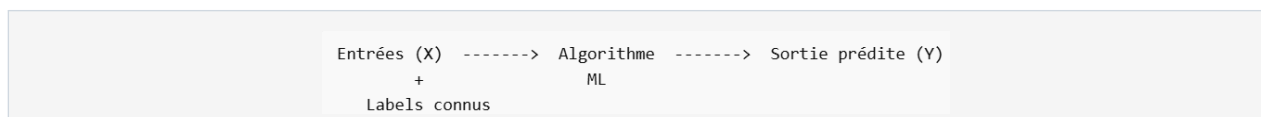


Figure 4 — Schéma de l'apprentissage supervisé

Classification — la sortie y est discrète : spam/non spam, type de maladie.

Régression — la sortie y est continue : prix d'une maison, consommation énergétique.

Algorithmes courants : régression linéaire/logistique, SVM, arbres de décision, forêts aléatoires, réseaux de neurones.

✉ **Fil conducteur — Spam** : Un filtre anti-spam supervisé est entraîné sur un ensemble d'emails étiquetés (spam = 1, non spam = 0). C'est un problème de **classification binaire**.

5.2. Apprentissage non supervisé

Pas d'étiquettes. L'objectif est de découvrir des structures cachées dans les données.

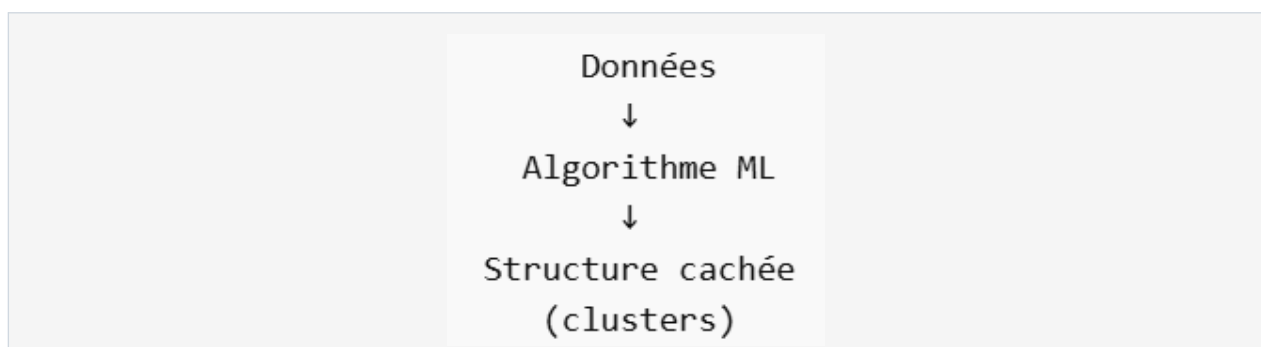


Figure 5 — Schéma de l'apprentissage non supervisé

- **Clustering** — regroupement en clusters (k-means, DBSCAN).
- **Réduction de dimension** — PCA, t-SNE, autoencodeurs.
- **Détection d'anomalies** — identifier les points aberrants (Isolation Forest).

✉ **Fil conducteur — Spam** : En non supervisé, on pourrait regrouper les emails en clusters (ex : publicité, newsletters, personnel, professionnel) sans étiquettes prédéfinies.

5.3. Apprentissage par renforcement

Un **agent** interagit avec un **environnement**, reçoit des récompenses ou des pénalités, et apprend une stratégie optimale.

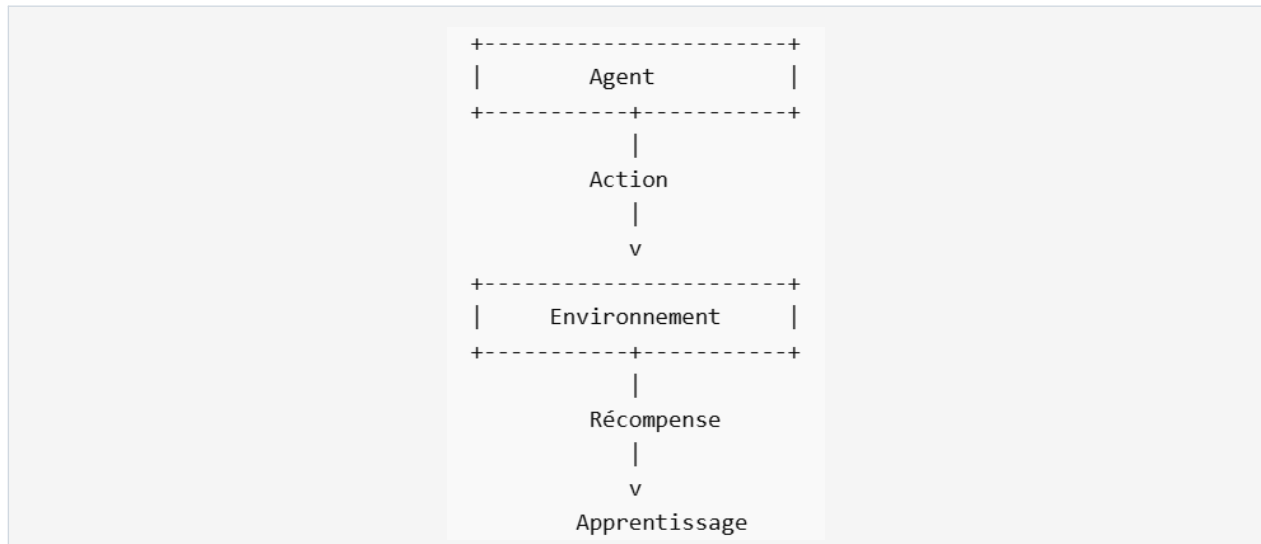


Figure 6 — Schéma de l'apprentissage par renforcement

Applications : robotique, jeux vidéo, véhicules autonomes, trading algorithmique.

Algorithmes : Q-learning, Deep Q Networks (DQN).

5.4. Apprentissage semi-supervisé et auto-supervisé

Semi-supervisé : combine peu de données étiquetées et beaucoup de données non étiquetées. Utile quand l'étiquetage est coûteux.

Auto-supervisé : le modèle génère ses propres labels (ex : prédire le mot suivant dans une phrase — BERT, GPT).

5.5. Synthèse

Type	Données	Objectif
Supervisé	(x, y) étiquetées	Prédire une sortie
Non supervisé	x seulement	Découvrir des structures
Par renforcement	Récompenses	Apprendre une stratégie
Semi-supervisé	Peu de (x,y) + beaucoup de x	Exploiter peu de labels
Auto-supervisé	Générés automatiquement	Apprendre des représentations

6. Concepts de base

6.1. Données et caractéristiques

Chaque exemple est un **vecteur de caractéristiques** $x = (x_1, \dots, x_n)$. Les données peuvent être structurées (tableaux) ou non structurées (texte, images). La sélection de caractéristiques pertinentes est appelée **feature engineering**.

✉ **Fil conducteur — Spam** : Pour un filtre anti-spam, les caractéristiques typiques d'un email sont : nombre de mots en majuscules, présence de mots-clés suspects (« gratuit », « urgent »), nombre de liens hypertextes, longueur du message, expéditeur connu ou non, etc.

Domaine	Exemples de caractéristiques
Médecine	Âge, tension artérielle, résultats d'analyses
Finance	Revenu, historique de crédit
Email (spam)	Nb mots majuscules, nb liens, longueur, mots-clés

6.2. Jeu de données et division

Ensemble	Rôle	Proportion typique
Entraînement	Apprentissage du modèle	60–70 %
Validation	Réglage des hyperparamètres	15–20 %
Test	Évaluation finale	15–20 %

6.3. Modèle et apprentissage

Un **modèle** est une fonction mathématique $Y = f(X, \theta)$. L'**apprentissage** consiste à ajuster les paramètres θ en minimisant une **fonction de perte** :

$$\min_{\theta} L(y, f(x, \theta))$$

— Structure d'un modèle ML

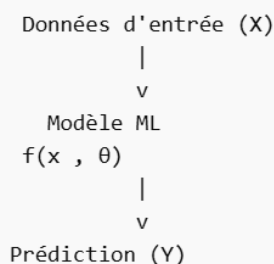


Figure 7 — Structure d'un modèle d'apprentissage automatique

6.4. Fonction de perte

Régression — MSE (erreur quadratique moyenne) :

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Exemple numérique : prédictions $\hat{y} = (200, 155, 310)$, valeurs réelles $y = (210, 150, 300)$.

$$MSE = [(210-200)^2 + (150-155)^2 + (300-310)^2] / 3 = [100 + 25 + 100] / 3 = \mathbf{75}.$$

Classification : entropie croisée, erreur de classification.

6.5. Généralisation, surapprentissage et sous-apprentissage

La **généralisation** est la capacité du modèle à bien prédire sur de nouvelles données.

Le **surapprentissage** (*overfitting*) survient quand le modèle mémorise les données d'entraînement.

Le **sous-apprentissage** (*underfitting*) survient quand le modèle est trop simple.

Cas	Entraînement	Test	Diagnostic
Sous-apprentissage	Erreur élevée	Erreur élevée	Modèle trop simple
Bon modèle	Erreur faible	Erreur faible	Bonne généralisation
Surapprentissage	Erreur très faible	Erreur élevée	Modèle trop complexe

✉ **Fil conducteur — Spam :** Un filtre anti-spam trop simple (ex : une seule règle sur un mot-clé) sera en sous-apprentissage. Un modèle trop complexe entraîné sur peu d'emails mémorisera des détails spécifiques et échouera sur de nouveaux spams.

6.6. Compromis biais-variance

Le **biais** mesure l'erreur systématique (modèle trop simple). La **variance** mesure la sensibilité aux fluctuations des données (modèle trop complexe).

L'erreur totale se décompose en : **Erreur = Biais² + Variance + Bruit irréductible.**

L'objectif est de trouver la complexité optimale minimisant l'erreur totale :

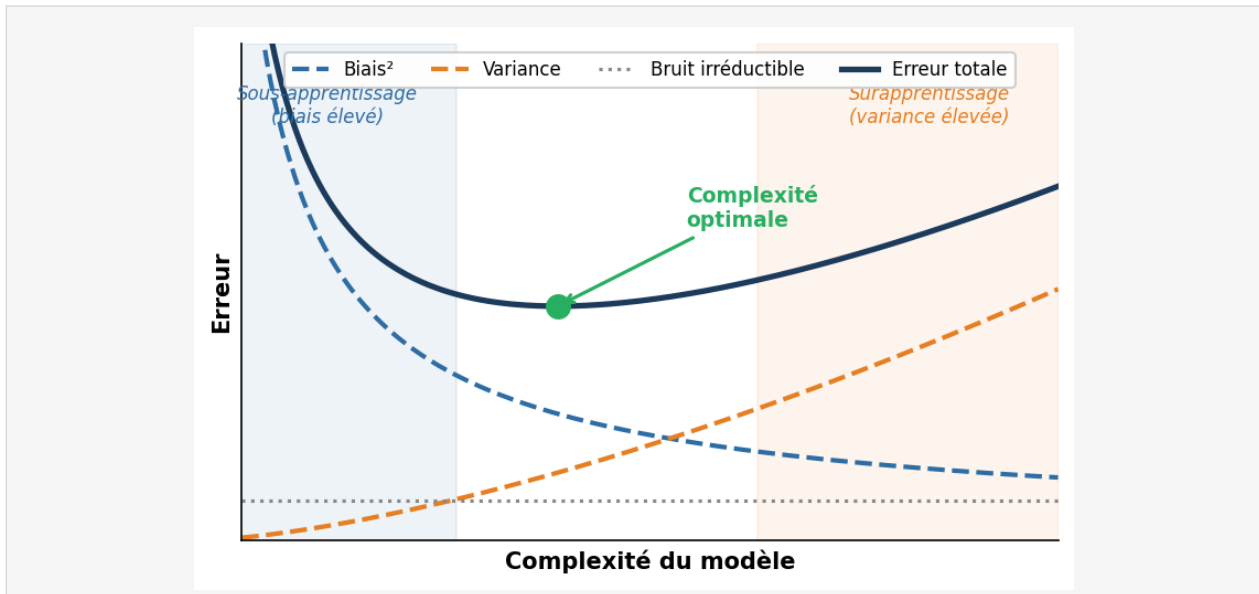


Figure 8 — Compromis biais-variance

6.7. Hyperparamètres

Paramètres définis **avant** l'apprentissage : nombre de voisins (k -NN), profondeur d'arbre, taux d'apprentissage. Optimisés par **validation croisée** (k -fold) : les données d'entraînement sont divisées en k sous-ensembles, et le modèle est évalué k fois en alternant le sous-ensemble de validation.

6.8. Métriques d'évaluation

Classification :

- *Accuracy* — taux de bonne classification.
- *Précision* — vrais positifs / prédits positifs. **Attention** : accuracy $\neq$ précision.
- *Rappel* — vrais positifs / réels positifs.
- *F1-score* — moyenne harmonique de la précision et du rappel.
- Matrice de confusion.

Régression : MSE, MAE, R^2 .

✉ **Fil conducteur — Spam** : Pour un filtre anti-spam, le *rappel* mesure la proportion de spams correctement détectés, tandis que la *précision* mesure la proportion d'emails classés spam qui sont réellement du spam. Un bon filtre doit éviter les faux positifs (emails légitimes classés spam).

7. Pipeline d'un projet d'apprentissage automatique

Le développement d'un système d'apprentissage automatique suit généralement une série d'étapes structurées permettant de transformer des données brutes en un modèle prédictif utilisable. Ce processus constitue ce que l'on appelle souvent le **pipeline**.

✉ **Fil conducteur — Spam : Objectif** : classer chaque email comme spam ou non spam. **Variable cible** : catégorie binaire (0 = non spam, 1 = spam). **Type** : apprentissage supervisé (classification binaire).

7.1. Définition du problème

La première étape consiste à formuler clairement le problème à résoudre et à déterminer si une approche par apprentissage automatique est pertinente. Il s'agit notamment de :

- identifier l'objectif de l'analyse,
- déterminer la variable à prédire,
- définir les données nécessaires,
- choisir le type d'apprentissage approprié (supervisé, non supervisé, etc.).

7.2. Collecte des données

Rassembler des données représentatives. Elles peuvent provenir de différentes sources :

- bases de données
- capteurs IoT
- logs systèmes
- images ou vidéos
- données web

La qualité et la diversité des données influencent directement les performances du modèle.

7.3. Préparation et nettoyage des données

Les données brutes doivent généralement être préparées avant d'être utilisées. Cette étape comprend :

- suppression des données aberrantes
- traitement des valeurs manquantes
- normalisation ou standardisation
- transformation des variables
- sélection des variables pertinentes
- gestion des déséquilibres de classes

Cette phase est souvent appelée **prétraitement des données**.

7.4. Extraction et sélection des caractéristiques

Les données doivent être représentées sous forme de caractéristiques (*features*) pertinentes pour l'apprentissage. Deux approches sont possibles :

Extraction de caractéristiques — Transformation des données pour obtenir une représentation informative. Exemples : histogrammes d'images, coefficients fréquentiels, embeddings de texte.

Sélection de caractéristiques — Choisir les variables les plus utiles afin de réduire la dimension du problème et améliorer les performances.

7.5. Division des données

Les données sont généralement divisées en plusieurs ensembles :

Ensemble	Rôle
Ensemble d'entraînement (training set)	Apprentissage du modèle
Ensemble de validation (validation set)	Ajustement des paramètres
Ensemble de test (test set)	Évaluation finale

Une pratique courante consiste à utiliser une répartition :

- 60–70 % entraînement
- 15–20 % validation
- 15–20 % test

7.6. Choix de l'algorithme d'apprentissage

L'algorithme dépend de la nature du problème et des données disponibles :

Problème	Algorithmes possibles
Classification	SVM, Random Forest, réseaux de neurones
Régression	Régression linéaire, régression ridge
Clustering	K-means, DBSCAN
Détection d'anomalies	Isolation Forest, autoencodeurs

7.7. Entraînement du modèle

Lors de cette étape, l'algorithme apprend les paramètres du modèle à partir des données d'entraînement. L'apprentissage consiste généralement à minimiser une fonction de coût (*loss function*).

$$\min_{\theta} L(y, f(x, \theta))$$

7.8. Évaluation du modèle

Une fois entraîné, le modèle est évalué afin de mesurer ses performances. Les métriques dépendent du type de problème.

Classification : accuracy, precision, recall, F1-score, matrice de confusion.

Régression : erreur quadratique moyenne (MSE), erreur absolue moyenne (MAE).

7.9. Optimisation et ajustement

Si les performances ne sont pas satisfaisantes, plusieurs actions peuvent être réalisées :

- ajustement des hyperparamètres (grid search, random search...)
- sélection d'autres caractéristiques
- augmentation des données
- choix d'un algorithme plus adapté

Cette phase peut nécessiter plusieurs itérations.

7.10. Déploiement du modèle

Une fois validé, le modèle peut être intégré dans un système réel : application web, système de détection d'intrusion, système de recommandation, plateforme industrielle. Le modèle peut alors produire des prédictions en temps réel.

7.11. Surveillance et mise à jour

Les performances du modèle doivent être surveillées dans le temps. Il peut être nécessaire de :

- ré-entraîner le modèle avec de nouvelles données
- corriger les dérives du modèle
- adapter le système aux nouvelles conditions

Le développement d'un système d'apprentissage automatique est un processus itératif combinant l'analyse des données, la modélisation mathématique et l'optimisation algorithmique.

✉ **Fil conducteur — Spam** : Un filtre anti-spam déployé doit être mis à jour régulièrement car les spammeurs adaptent constamment leurs techniques (concept de *dérive conceptuelle*).

7.12. Technologies et plateformes

Technologie	Usage
Python	Langage principal de l'apprentissage automatique
Scikit-learn	Algorithmes classiques (SVM, RF, k-NN, etc.)
TensorFlow	Apprentissage profond industriel
PyTorch	Recherche et apprentissage profond
Keras	Interface simplifiée pour réseaux de neurones

7.13. Exemple de bout en bout : classification du dataset Iris

Voici un exemple complet illustrant le pipeline avec Python et Scikit-learn, appliqué à la classification du dataset Iris :

```

# 1) Importation des bibliothèques
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
                                classification_report

# 2) Chargement du dataset
iris = load_iris()
X = iris.data          # Features (4 variables)
y = iris.target        # Labels (3 classes)

# 3) Création d'un DataFrame
df = pd.DataFrame(X, columns=iris.feature_names)
df['classe'] = y
print(df.head())
print(df.describe())

# 4) Visualisation simple
plt.figure(figsize=(7, 5))
for classe in np.unique(y):
    plt.scatter(X[y==classe, 0], X[y==classe, 2],
                label=iris.target_names[classe])
plt.xlabel(iris.feature_names[0])
plt.ylabel(iris.feature_names[2])
plt.legend(); plt.grid(True); plt.show()

# 5) Division train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 6) Création et entraînement du modèle
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# 7) Prédiction
y_pred = model.predict(X_test)

# 8) Évaluation du modèle
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred,
                               target_names=iris.target_names)
print(f'Accuracy = {acc:.4f}')
print('Matrice de confusion : ', cm)
print(report)

# 9) Test sur une nouvelle fleur
nouvelle_fleur = [[5.1, 3.5, 1.4, 0.2]]
pred = model.predict(nouvelle_fleur)[0]
print('Classe prédite : ', iris.target_names[pred])

```

Figure 9 — Pipeline complet Scikit-learn : classification du dataset Iris

8. Limites et défis de l'apprentissage automatique

Malgré ses succès, l'apprentissage automatique présente des limites importantes :

8.1. Qualité des données

Les performances dépendent fortement de la qualité et de la quantité des données. « *Garbage in, garbage out* ». La collecte, le nettoyage et l'annotation constituent souvent l'étape la plus coûteuse.

8.2. Interprétabilité

Les modèles complexes (apprentissage profond) sont des « boîtes noires ». L'**IA explicable** (XAI) développe des méthodes de transparence : LIME, SHAP, cartes d'attention.

8.3. Biais algorithmiques

Un modèle entraîné sur des données biaisées reproduit et amplifie ces biais. La détection et la correction des biais sont des enjeux éthiques et techniques majeurs.

8.4. Coût computationnel

Les grands modèles de langage nécessitent des ressources considérables (GPU/TPU). Techniques d'optimisation : quantification, distillation, élagage (*pruning*).

8.5. Sécurité et confidentialité

Attaques adversariales : des perturbations imperceptibles peuvent tromper les modèles.

Confidentialité : l'entraînement sur données sensibles pose des enjeux réglementaires (RGPD). Solutions : apprentissage fédéré, confidentialité différentielle.